

1. Project Overview

LibriTube is a browser extension (Firefox / Zen Browser) that fundamentally changes how you interact with YouTube. Instead of watching videos, you *read* them. The extension locks the video player and presents a rich, PDF-reader-style modal containing:

- A concise **summary**
- **Key points** (positive and negative) in bullet form
- A **harsh critique & evaluation** with direct citations from the transcript
- The **full formatted transcript**, turned into readable prose

The core goal: **Accelerate research and learning for people who absorb information faster by reading than by watching.**

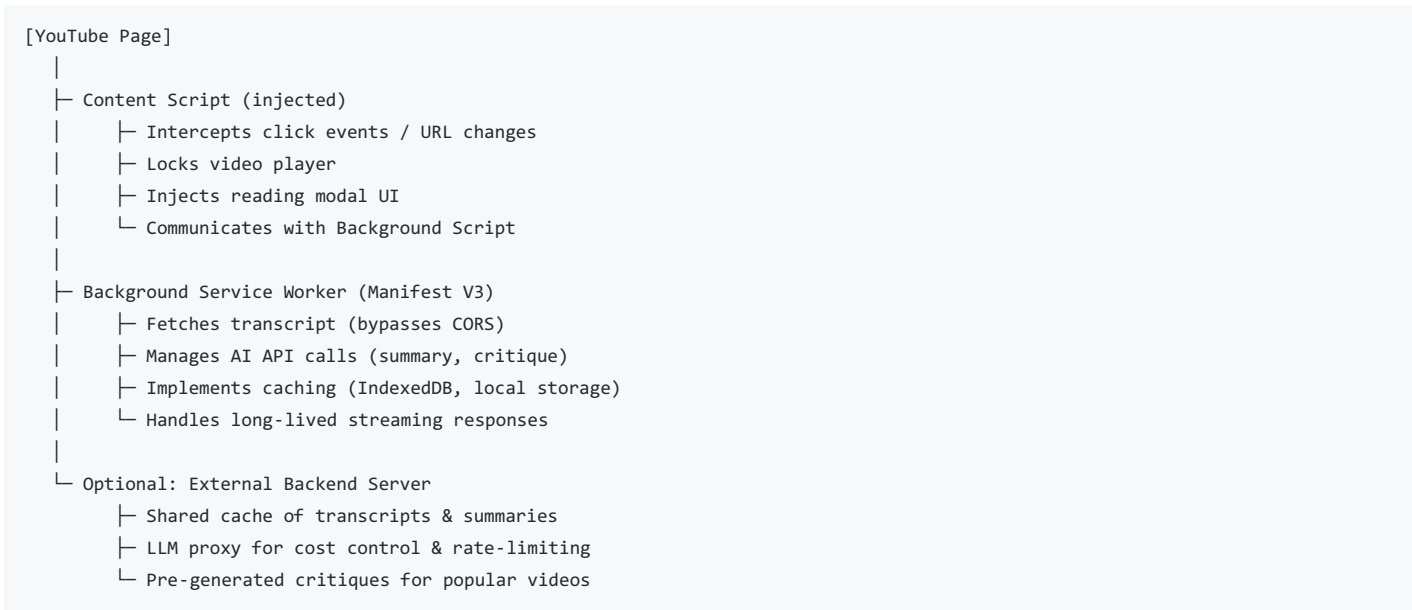
2. Core Features

1. **Click-to-Read** – On YouTube homepage, search results, or channel pages, clicking a video thumbnail or title does not play the video. Instead, it pauses and locks the video (if on a watch page) and immediately opens the reading modal.
2. **Video Lock** – The YouTube `<video>` element is paused and made non-interactive (`pointer-events: none`, playback controls hidden), eliminating the distraction of auto-play.
3. **PDF-Reader-Styled Modal** – A full-screen overlay with a clean, paginated layout, table of contents, dark/light mode, and adjustable font size – mimicking a digital book or PDF reader.
4. **Intelligent Content Generation** – Summary, key points, harsh judgment, and full script, all generated using the transcript.
5. **Watch-Later Toggle** – A button to “Unlock & Watch” the video if the reading convinces you it’s worth watching.

3. User Flow

1. User browses YouTube normally.
2. Clicks any video.
3. Video page loads (or the current page is intercepted), the video is instantly paused, and the modal slides in.
4. A loading skeleton appears while transcript is fetched and AI content is generated (streaming if possible).
5. The reader modal populates and the user reads seamlessly.
6. User can close the modal to return to the video page (video remains locked), or press “Watch” to unlock.

4. High-Level Technical Architecture



5. Key Technologies & Tools

Layer	Technology	Purpose
Extension		

Framework Layer	WebExtensions API (Manifest V3) Technology	Cross-browser compatibility (Firefox, Zen) Purpose
Frontend UI	React / Preact (embedded as content script), Styled Components / Tailwind	Lightweight, responsive reader modal
Video Lock	<code>video.pause()</code> , CSS <code>pointer-events: none</code> , overlay element	Prevent playback and UI interaction
Transcript Retrieval	YouTube InnerTube API (<code>/youtubei/v1/player</code>), DOM scraping (<code>ytInitialPlayerResponse</code>)	Fetch timed captions without official API key
AI Inference	OpenAI GPT-4o / Claude 3.5 Sonnet (via API), or local WebLLM (e.g., Llama 3.2 via WebGPU)	Summarisation, bullet extraction, harsh critique
Citation Mapping	Prompt engineering + post-processing	LLM returns text with <code>[timestamp]</code> markers; client renders clickable links
Caching & Storage	IndexedDB, <code>browser.storage.local</code>	Store transcripts, summaries, user preferences, reduce repeated API calls
Streaming & Performance	Server-Sent Events (SSE) or <code>ReadableStream</code>	Show progressive results, masking latency
Build & Packaging	Vite, web-ext, TypeScript	Fast development, type safety

6. Data Pipeline & AI Strategy

6.1 Obtaining the Transcript

- 1. Extract video ID from the URL or intercepted click.
- 2. Background script fetches the YouTube watch page or InnerTube API endpoint.
- 3. Parse the `captions` object inside `playerResponse` → select the desired language track (prefer manual over auto-generated).
- 4. Fetch the actual caption XML/JSON from the `baseUr1`.
- 5. Clean and structure the transcript into an array of `{text, start, duration}` segments.

Problem: CORS restrictions, YouTube rate-limiting.
Fix: Background script has no CORS limitations; use exponential backoff and cache transcripts aggressively.

6.2 Generating Summary & Key Points

- Send the full transcript (truncated if necessary) to an LLM with a prompt like:

```
You are a mercilessly honest editor. Summarize this video transcript in 3-4 sentences. Then list the 5 most important positive
```
- **Citation format:** LLM outputs `[MM:SS]` or `[start_seconds]`; the UI parses and makes them clickable.

6.3 Harsh Critique / Evaluation

- A second prompt:

```
Now give a brutally honest, sarcastic critique of this video. Attack logical fallacies, factual errors, bias, poor structure,
```

6.4 Full Script Formatting

- Convert raw transcript segments into readable paragraphs: merge sentences, insert paragraph breaks where natural pauses occur (>1.5s gaps), add speaker labels if present.

7. Addressing Common Problems & Latency

7.1 Latency – The Core Challenge

- **Instant Skeleton UI** – Show placeholder immediately with progress bar.
- **Streaming Responses** – Use SSE to progressively display content.

- **Pre-fetching on Hover** – When hovering a thumbnail >500ms, fetch transcript and queue AI job.
- **Local Caching** – Store all generated content in IndexedDB per video ID; subsequent loads are instant.
- **Community Cache (Optional)** – Lightweight backend for popular videos.
- **Lightweight “Express” Mode** – Local extractive summary (first/last sentences + TF-IDF key phrases) in <1s while the full AI loads.
- **Edge Computing** – Deploy LLM on Cloudflare Workers/Fly.io close to users.

7.2 Transcript Access Reliability

- Multiple fallback strategies:
 1. `ytInitialPlayerResponse` from watch page HTML.
 2. InnerTube API with extracted client key.
 3. Third-party transcript service as last resort.
- Use mutation observers to know when the player response is ready.

7.3 Single Page Application (SPA) Navigation

- YouTube uses SPA. Intercept `history` API (`pushState` / `replaceState`) and listen to `yt-navigate-finish` custom events to re-apply the lock and modal on video changes.

7.4 Video Lock Tampering

- A user could disable the lock via DevTools – this is acceptable. The lock is a nudge, not DRM. “Watch” button easily reverses it.

7.5 Privacy & Data Usage

- Transparency on data sent; privacy policy.
- Two inference modes: **Cloud AI** (fast, optional API key) and **Local AI** (WebLLM, data never leaves the device).
- Optional backend proxy with no-log policy.

7.6 Cost Management

- Aggressive caching (local + optional shared).
- Use cheap model for summary, powerful only for critique, or skip critique if user prefers.
- Throttle requests.

7.7 Handling Edge Cases: Tabs, Refreshes, and Playlists

7.7.1 Opening a Video in a New Tab / Middle-Click / Page Refresh

When a user opens a video in a new tab (Ctrl+click, middle-click) or refreshes an already-open watch page, the extension must:

- **Prevent a visual flash of the video player.**
Inject CSS at `document_start` that sets `opacity: 0` on the `<video>` and its containers until the lock is applied. On `DOMContentLoaded`, pause the video, apply the lock, then restore visibility. This eliminates the “video starts playing for a split second” glitch.
- **Re-fetch state from scratch.**
Extract the video ID from the URL and begin the full pipeline. Cached data from IndexedDB is used if available, making repeated opens feel instantaneous.

7.7.2 Playlist Autoplay & Manual “Next Video”

YouTube playlists change the video content without a full page reload. The extension must:

- **Detect video ID changes reliably.**
 - Primary: Listen to `yt-navigate-finish` custom event.
 - Fallback: Use a `MutationObserver` on the `<video>` element’s `src` attribute or `document.title`.
- **Immediately lock the new video.**
On detection, pause the new video, re-apply the lock, dismiss the old modal, and show a fresh loading placeholder for the new video’s content.
- **Pre-fetch for seamless reading.**
When a playlist is detected (`&list=` in the URL), pre-emptively fetch transcripts and trigger AI generation for the next 1-2 videos. Results are stored in IndexedDB, so advancing to the next video loads the reading material instantly.
- **Preserving “Watch” functionality.**
“Unlock & Watch” applies only to the current video. When a playlist advances, the new video automatically locks again, respecting the user’s default reading mode.

7.7.3 Cross-Case Robustness

- **Respecting user intent:** If the user explicitly presses Play (via “Watch” button), the lock is removed and autoplay resumes normally.
- **Restoring after network interruption:** A refresh simply reloads the modal for the same video; no reading state is lost.
- **Bookmark handling:** Opening a bookmarked watch URL results in the full reading experience, just like any new tab.

8. User Interface Design – The PDF Reader Modal

- **Navigation:** Left sidebar with Table of Contents (Summary, Key Points, Critique, Full Script).
 - **Reading View:** Distraction-free, paginated or scrollable, adjustable font (serif/sans, size, line height).
 - **Citations:** Blue underlined timestamps; clicking them temporarily reveals the video player overlay at that exact moment (picture-in-picture style).
 - **Dark/Light mode:** Syncs with YouTube's theme automatically.
 - **Keyboard shortcuts:** `Esc` to close, arrow keys to turn pages, `w` to unlock and watch.
-

9. Algorithms & AI Prompts (Summary)

- **Transcript extraction:** DOM parsing + JSON navigation, fallback chain.
 - **Summarisation:** LLM zero-shot/few-shot with structured output (JSON mode).
 - **Key Points:** LLM outputs two arrays (strengths , weaknesses) each containing {point, quote, timestamp} .
 - **Critique Generation:** Persona-based prompt ("ruthless book critic"), outputting markdown with citation links.
 - **Citation Verification:** Post-processing ensures timestamps exist; if not, LLM is prompted to correct.
-

10. Potential Future Enhancements

- **"Shelf" feature** to save read-videos as a personal, searchable library.
 - **Speed-reading mode** (RSVP).
 - **Collaborative annotations** directly on transcript segments.
 - **YouTube Shorts support** with micro-summaries.
-

11. Conclusion

LibroTube reimagines YouTube as a vast digital library where every video becomes a "book" you can skim, analyse, and judge without ever pressing play. It now flawlessly handles **direct link opens, page refreshes, and seamless playlist transitions**, ensuring the reading experience is never disrupted by YouTube's dynamic behaviour. By combining modern browser APIs, robust transcript retrieval, and state-of-the-art LLMs with aggressive caching and instant-response strategies, the transition from video to text feels nearly invisible—even across tabs and autoplay queues.